DÉTECTION ET CLASSIFICATION

D'AÉRONEFS MILITAIRES

À PARTIR D'IMAGES AÉRIENNES EN ZONE DE CONFLIT

Alaa CHACHA
Hajar GUEMILI

Algorithmes utilisés :

YOLOv8s	Random Forest	CNN ResNet50	EfficientNetV2
Détection	Classification	Classification	Classification

Aperçu du projet :

Ce projet développe un pipeline complet de Computer Vision pour détecter et identifier automatiquement les aéronefs militaires dans des images aériennes. Il combine une approche de détection par Deep Learning (YOLOv8) et deux approches de classification : une méthode classique (Random Forest) et un réseau CNN (ResNet50).

PROBLÉMATIQUE & OBJECTIFS

Contexte

Dans un contexte de guerre, la surveillance aérienne est essentielle pour la prise de décision stratégique. La détection manuelle est lente, coûteuse et sujette à des erreurs humaines. L'essor du Deep Learning ouvre de nouvelles perspectives pour automatiser cette tâche.

Problématique

Comment concevoir un système capable de détecter automatiquement les aéronefs militaires dans des images aériennes et de classifier leur type avec précision, malgré le bruit, les variations d'éclairage, la distance et les angles de vue ?

Objectifs :

01 — **Détecter & localiser**

Identifier la position des aéronefs (bounding box)

02 — **Classifier**

Identifier le type parmi 101 classes militaires

03 — **Comparer les modèles**

Random Forest vs CNN ResNet50 vs YOLOv8

04 — **Évaluer**

mAP, accuracy, F1-score, temps réel

BASE DE DONNÉES — MILITARY AIRCRAFT DETECTION DATASET

41 441	23 126	101	1.79
Annotations	Images	Classes	Avions/image

Structure du dataset :

dataset/

Images brutes .jpg + CSV individuels → utilisé pour la DÉTECTION (YOLOv8)

crop/

Images découpées par classe (101 dossiers) → utilisé pour la CLASSIFICATION

annotated_samples/

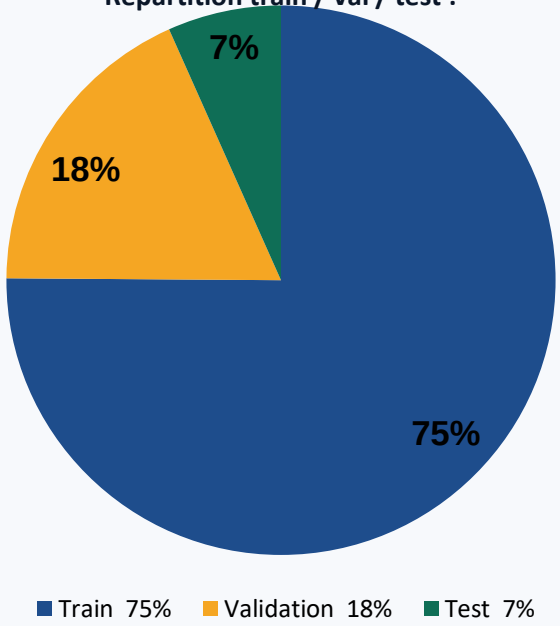
Images avec bbox dessinées → vérification visuelle uniquement

labels_with_split.csv

41 441 lignes : filename · width · height · class · xmin · ymin · xmax · ymax · split

BASE DE DONNÉES — DISTRIBUTION & RÉPARTITION

Répartition train / val / test :



Déséquilibre des classes :

F16
2 061 annotations

F18
1 796 annotations

C130
1 590 annotations

MQ20
11 annotations

WZ9
15 annotations

V280
39 annotations

Impact du déséquilibre :

Les classes sous-représentées (WZ9=15, MQ20=11) seront mal apprises par le modèle. Solution : augmentation de données ciblée sur ces classes.

PREPROCESSING — NETTOYAGE, ENCODAGE & CONVERSION

Étape 1

Nettoyage des bbox invalides

Bbox inversées : xmax < xmin ou ymax < ymin → rectangle impossible

Bbox hors image : coordonnées négatives ou dépassant width/height

Bbox trop petites : largeur ou hauteur < 10 pixels

Résultat : 59 annotations supprimées sur 41 441

Étape 2

Encodage des classes

YOLOv8 ne comprend pas les noms textuels

Dictionnaire : A10=0, F16=31, Rafale=69, Z19=100

Fichier classes.txt sauvegardé pour décodage après prédiction

101 classes encodées de 0 à 100

Étape 3

Conversion format Pascal VOC → YOLO

Pascal VOC : xmin, ymin, xmax, ymax (coordonnées absolues en pixels)

YOLO : cx, cy, bw, bh (normalisées entre 0 et 1)

cx = (xmin+xmax)/2/W · cy = (ymin+ymax)/2/H

Normalisation : indépendance vis-à-vis de la résolution de l'image

PREPROCESSING — STRUCTURE YOLO & FICHIER YAML

Étape 4

Création structure dossiers

train/images/ + train/labels/ → 17 326 paires

val/images/ + val/labels/ → 4 256 paires

test/images/ + test/labels/ → 1 544 paires

Total : 23 126 images copiées + 23 126 fichiers .txt créés

Étape 5

aircraft .yaml

path: chemin racine du dataset YOLO

train / val / test : sous-dossiers contenant les images

nc: 101 → nombre de classes à détecter

Sans ce fichier YAML → YOLOv8 ne peut pas démarrer l'entraînement

Format d'un fichier label .txt (une ligne = un aéronef) :

```
31    0.294878    0.582720    0.028150    0.033700

F16      cx=29.4%      cy=58.2%      bw=2.8%      bh=3.3%      (normalisé entre 0 et 1)
```

YOLOV8 — ARCHITECTURE & FONCTIONNEMENT

YOLOv8 (You Only Look Once) est un détecteur one-stage : il analyse l'image EN UNE SEULE PASSE pour détecter tous les objets simultanément. Il prédit la position (bbox) ET la classe en même temps.

Architecture :



Fichiers nécessaires pour l'entraînement :

`yolov8s.pt`	Modèle pré-entraîné sur COCO (Transfer Learning) — 21.5 MB
`aircraft.yaml`	Configuration : chemins images, nc=101, noms des classes
`yolo_dataset/`	17326+4256+1544 images + labels .txt correspondants
`classes.txt`	Liste des 101 classes pour décoder les prédictions numériques

YOLOV8 — ENTRAÎNEMENT & PARAMÈTRES

YOLOv8s Modèle	**50** Epochs	**16** Batch
640×640 Image	**Tesla T4** GPU	**~5h** Durée

Cycle d'apprentissage (1 epoch) :

17 326 images train → **Prédiction bbox + classe** → **Calcul Loss** → **Mise à jour 11M paramètres** → **Répéter 50 fois**

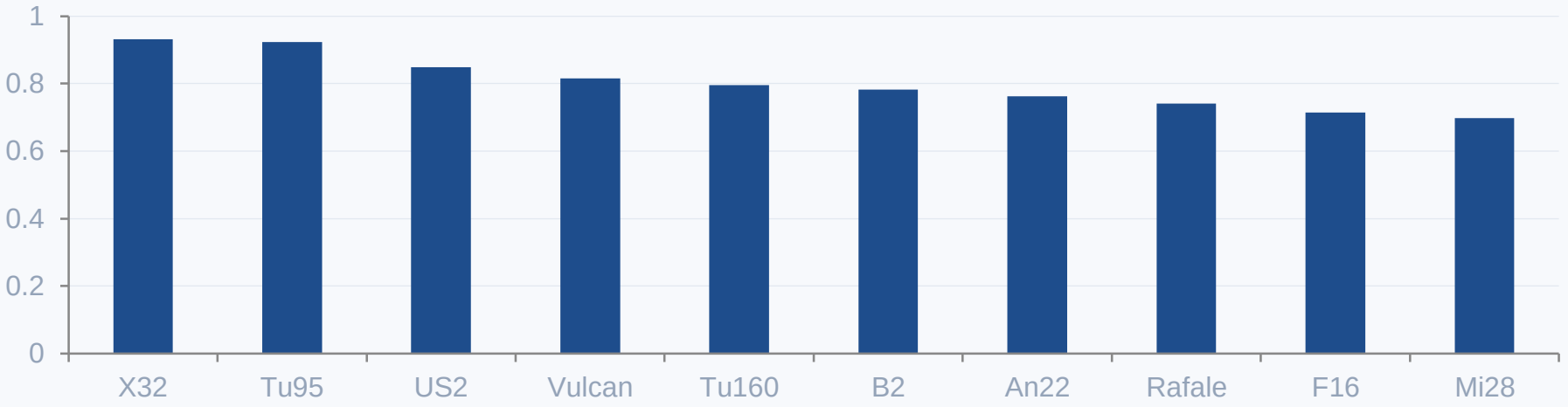
Transfer Learning :

YOLOv8s pré-entraîné sur COCO (120 000 images, 80 classes générales) connaît déjà les formes, contours et textures. On lui apprend EN PLUS à reconnaître nos 101 types d'aéronefs militaires. Résultat : convergence beaucoup plus rapide qu'un entraînement from scratch.

YOLOV8 — RÉSULTATS & MÉTRIQUES

0.743	0.821	0.698	8.6ms
mAP@0.5	**Précision**	**Rappel**	**Vitesse**
Précision globale	Détections correctes	Aéronefs trouvés	Temps réel / image

Top 10 classes — mAP@0.5 :



YOLOV8 — VISUALISATION DES DÉTECTIONS



US2

Score : 0.94

Avion amphibie japonais

Base navale



Mirage2000

Score : 0.62

Chasseur français

Base militaire



An22

Score : 0.91

Avion cargo russe

Vol en altitude

Logs de prédiction :

1537/1544 → 5 F16s · 1538 → 1 B1 · 1541 → 1 Mi28 · 1542 → 1 EF2000 + 1 Rafale · 1544 → 1 F35 + 1 Su57

BILAN — PARTIE DÉTECTION YOLOV8

✓ **Pipeline preprocessing complet**

41 382 annotations nettoyées · 23 126 images converties au format YOLO · structure train/val/test

✓ **Entraînement réussi**

50 epochs sur Tesla T4 · 11 164 671 paramètres · best.pt sauvegardé (21.5 MB)

✓ **Performances solides**

mAP@0.5 = 0.743 · Précision = 0.821 · Rappel = 0.698 · 8.6ms par image

✓ **Détection temps réel confirmée**

8.6ms/image compatible avec la surveillance en temps réel · détection multi-aéronefs par image

! **Limite : déséquilibre des classes**

Classes minoritaires (WZ9=15, MQ20=11) → mAP faible · correction : augmentation de données

SECTION 02

Classification des aéronefs

Random Forest vs CNN ResNet50

101	crop/	2
Classes	Dossier source	Modèles comparés

CLASSIFICATION — RANDOM FOREST (APPROCHE CLASSIQUE)

Approche : extraction manuelle de features (HOG + couleur + texture) → vecteur de 1816 dimensions → Random Forest



Résultats :

16.29%	36.15%	6837 FPS	556.7s
Accuracy Top-1	**Accuracy Top-5**	**Vitesse**	**Temps entraîn.**
Performance limitée	Meilleure Top-5	Très rapide	9 minutes

Exemple Top-5 RF : F35 (6.30%) · F18 (5.15%) · J20 (4.33%) · F15 (3.87%) · V22 (3.36%)

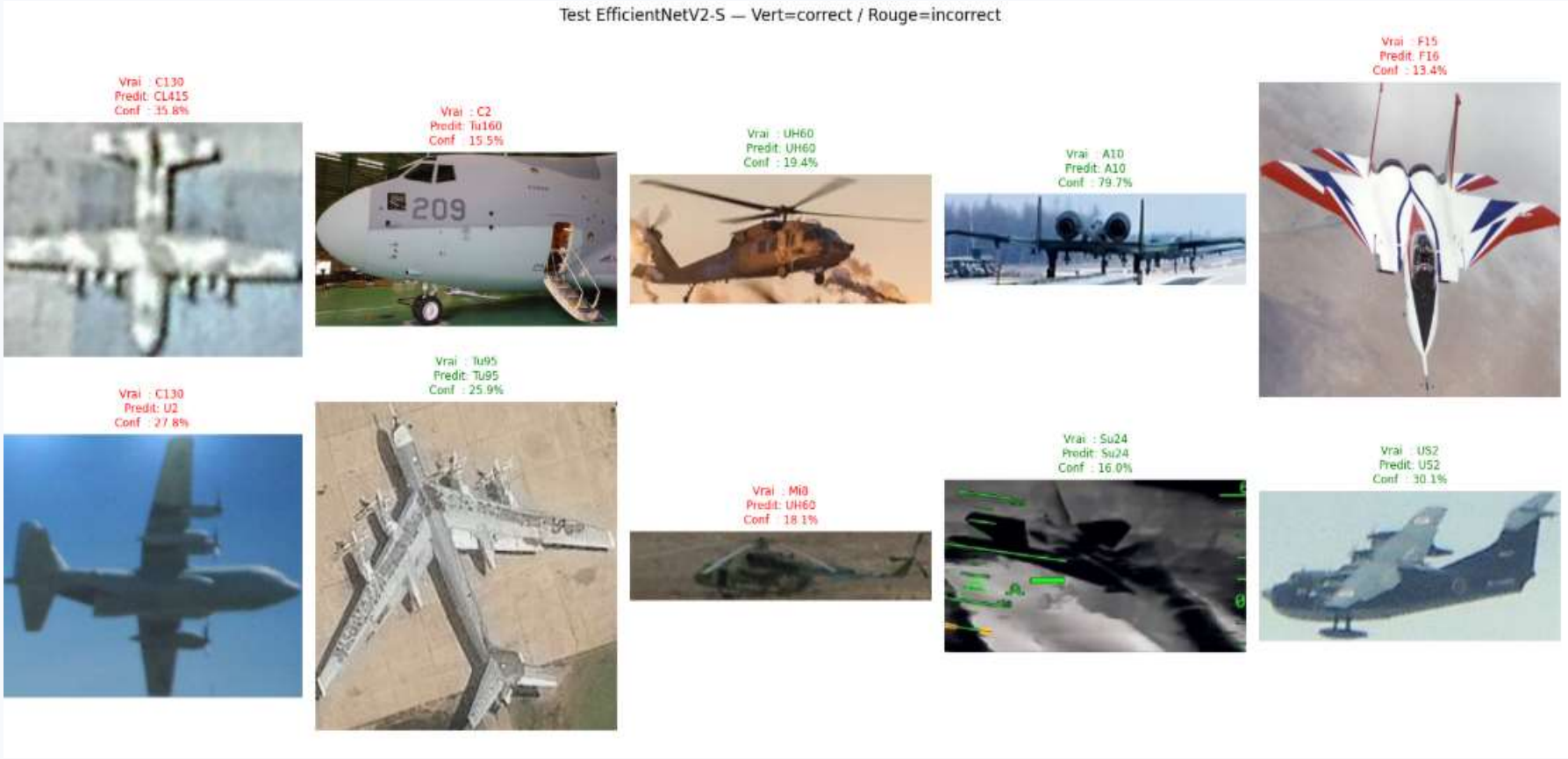
Confiance très faible sur toutes les classes → modèle non discriminant → limite de l'approche classique

CLASSIFICATION — RANDOM FOREST (APPROCHE CLASSIQUE)



CLASSIFICATION — EfficientNetV2

```
================================================
RESULTATS : EfficientNetV2-S sur Test
================================================
  Accuracy Top-1 : 0.5096  (50.96%)
  Accuracy Top-5 : 0.7970 (79.70%)
```

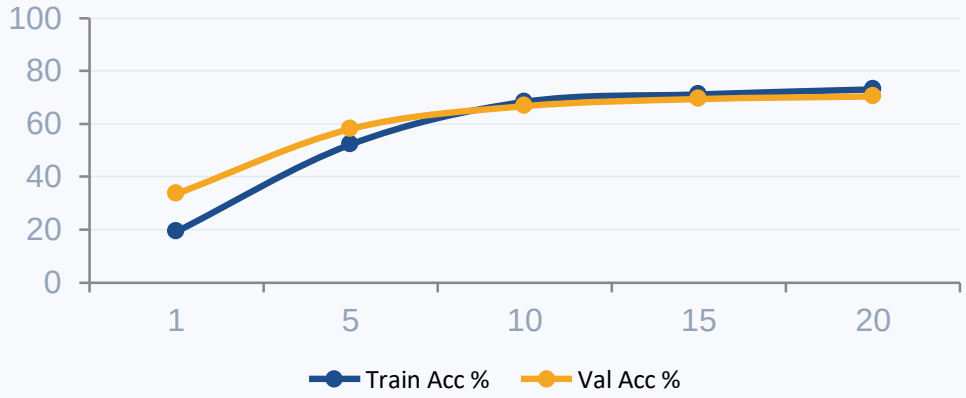


CLASSIFICATION — CNN RESNET50 (DEEP LEARNING)

ResNet50 pré-entraîné sur ImageNet (1M images, 1000 classes) → Fine-tuning sur 101 classes d'aéronefs · 20 epochs · GPU Tesla T4

Architecture ResNet50 :





Top-1 Accuracy
70.49%

Top-5 Accuracy
89.27%

Vitesse
122.6 FPS

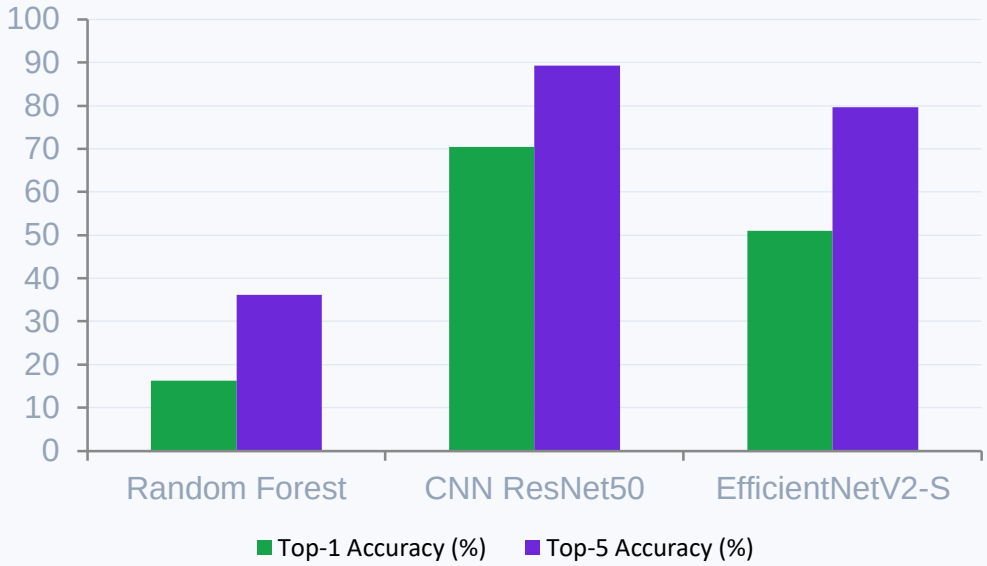
Exemple : F14 → 99.87% de confiance !

CLASSIFICATION — CNN RESNET50 (DEEP LEARNING)

```
TEST CNN sur quelques exemples
======================================================
```



COMPARAISON FINALE — 3 MODÈLES DE CLASSIFICATION



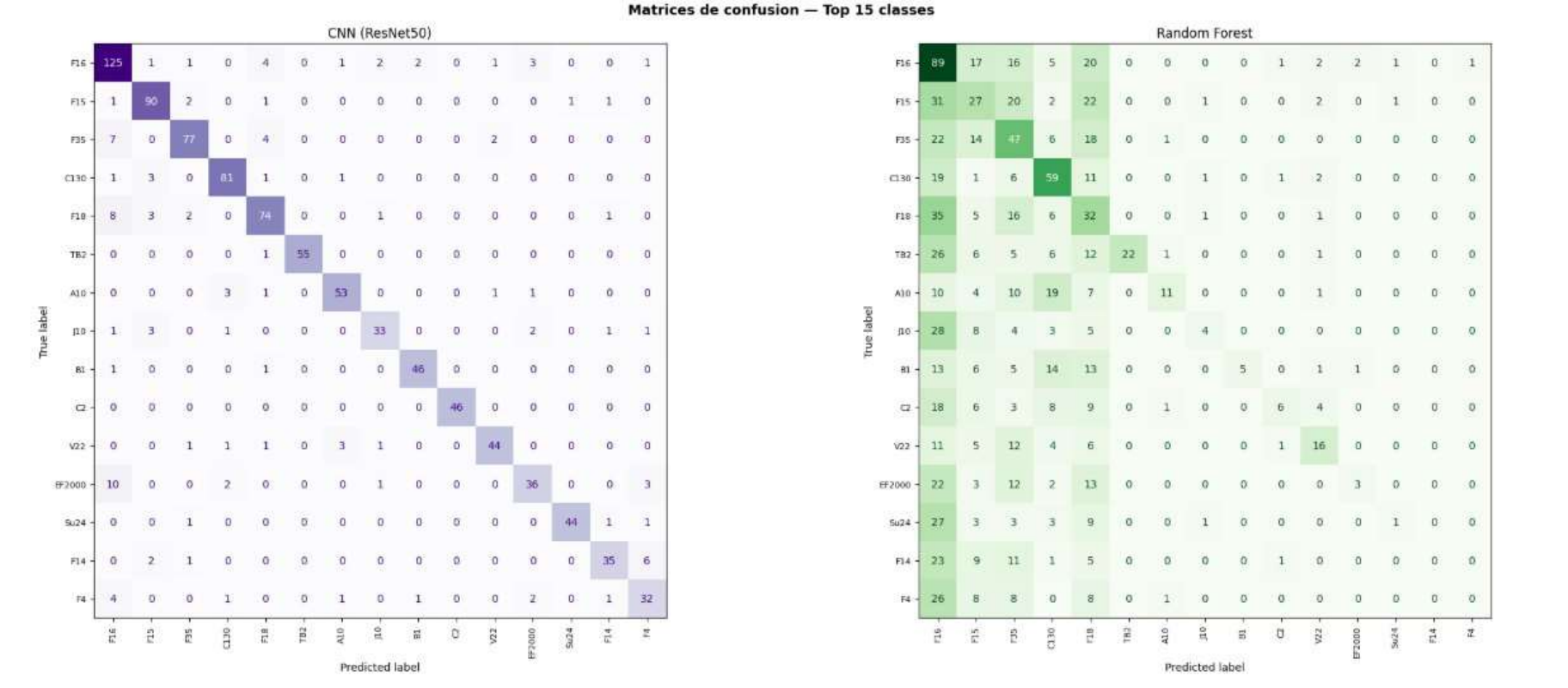

```
============================================================
  COMPARAISON FINALE — 3 MODELES DE CLASSIFICATION
============================================================
Modele                    Top-1    Top-5      FPS
------------------------------------------------------------
CNN (ResNet50)          70.49%   89.27%    122.6
Random Forest           16.29%   36.15%   6837.4
EfficientNetV2-S        50.96%   79.70%    172.2
============================================================
```

Tableau comparatif :

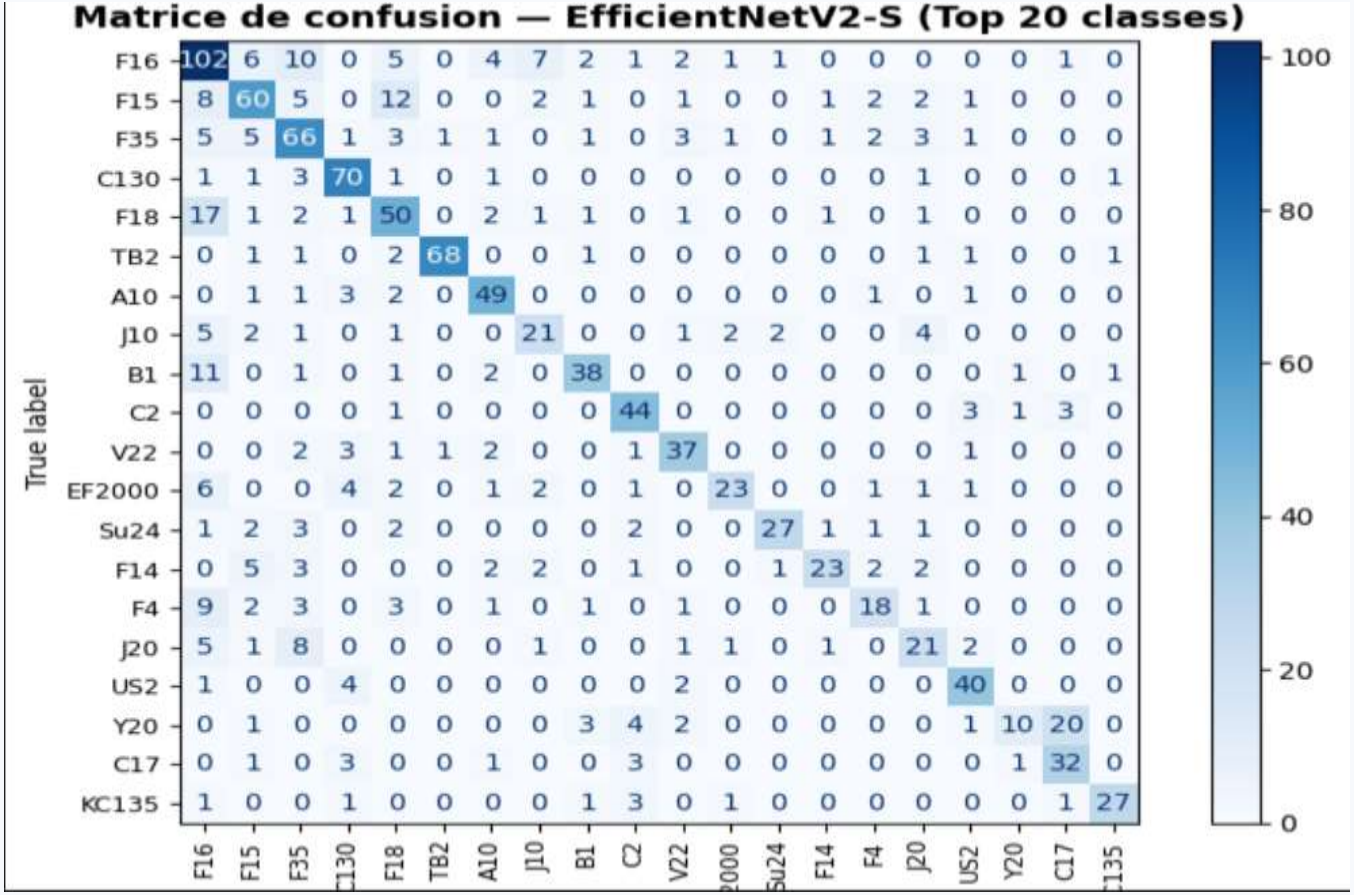
Critère	Random Forest	ResNet50	EfficientNetV2
Top-1 Acc.	16.29%	70.49%	50.96%
Top-5 Acc.	36.15%	89.27%	79.70%
Vitesse	6837 FPS	122.6 FPS	172.2 FPS
Entraîn.	556.7s	~20 epochs	~15 epochs
Approche	HOG + ML	CNN DL	CNN DL
Recommandé	Non	Oui	Oui

CLASSIFICATION — MATRICES DE CONFUSION

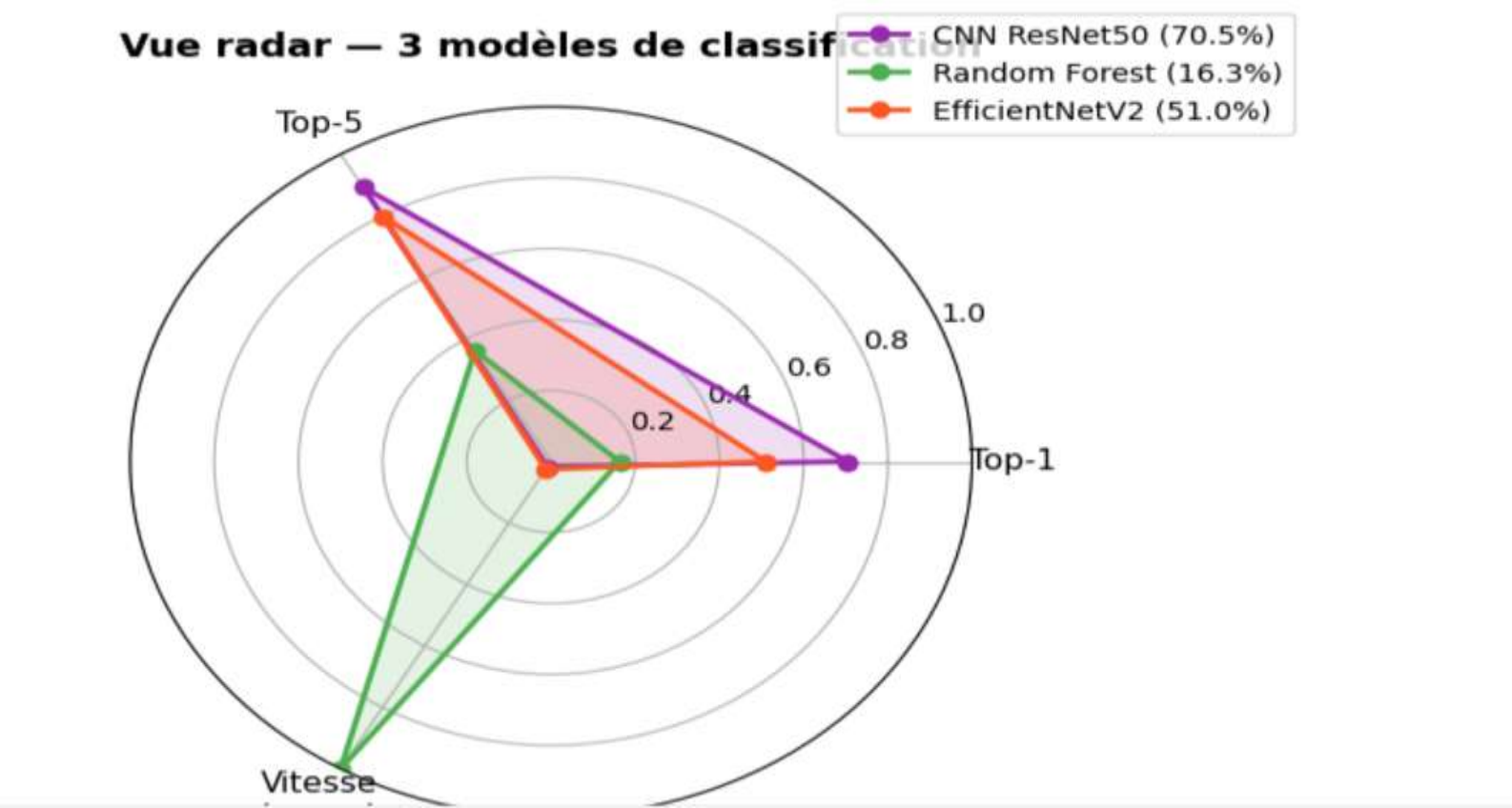
Top 15 classes — CNN ResNet50 (gauche) vs Random Forest (droite) — Diagonale = prédictions correctes



CLASSIFICATION — MATRICES DE CONFUSION



COMPARAISON — VUE RADAR CNN vs RANDOM FOREST vs EfficientNetV2



MERCI POUR VOTRE ATTENTION